Operating Systems Project Report

CPU Priority Scheduling Simulation

Group 10 — Course Code: 012012500105

Instructor: Mai Ngọc Châu

Repository: https://github.com/Hhanguu/PriorityScheduling
Topic Code: OS_SCH_03
Date: November 11, 2026

Table of Contents

1. Introduction
2. Problem Analysis
3. System Design
4. Implementation
5. Testing
6. Performance Benchmarks
7. Results and Discussion
8. Conclusion
9. Member Contributions

1. Introduction

1.1 Background

CPU scheduling is a fundamental mechanism in operating systems that determines which process gets CPU time and for how long. Priority scheduling assigns a numeric priority to each process; the CPU is always given to the highest-priority ready process (lowest priority number = highest priority).

This project implements and compares two variants of priority scheduling:

- **Non-preemptive Priority Scheduling** — once a process is selected to run, it executes until it finishes, regardless of new arrivals.
- **Preemptive Priority Scheduling (SRPT-style)** — if a newly arrived process has strictly higher priority than the running one, it preempts the CPU immediately.

1.2 Objectives

1. Implement both algorithms correctly and efficiently in C++17.
2. Build a Qt6 desktop GUI that visualizes results as a Gantt chart and comparison table.
3. Read input from CSV; export five result CSV files per run.
4. Measure performance up to 2,000,000 processes and confirm O(n log n) behavior.
5. Cover correctness with an automated test suite (unit + integration + GUI).

1.3 Team

#	Full Name	Student ID	Responsibility
1	Hưng (Leader)	—	Architecture, CMake, integration tests, benchmark, GUI review
2	Kiệt NP	—	NonPreemptiveScheduler, N01–N18 unit tests
3	Vĩ P	—	PreemptiveScheduler, P01–P18 unit tests
4	Minh Hưng GUI	—	MainWindow, GanttChart, ProcessTableModel, GUI tests
5	Lan CSV	—	CsvHandler, C01–C25 unit tests

2. Problem Analysis

2.1 Input and Output Specification

Input: CSV file with header `pid,arrival,burst,priority`

```
pid,arrival,burst,priority
1,0,5,3
2,1,3,2
3,2,8,1
```

Constraints (enforced by `CsvHandler`):

Field	Rule
`pid`	Integer > 0, unique within file
`arrival`	Long long ≥ 0
`burst`	Long long > 0
`priority`	Integer, lower = higher priority

Output: Five CSV files per run (when both algorithms are selected):

File	Content
`np_result.csv`	CT, TAT, WT, RT for each process (non-preemptive)
`np_gantt.csv`	Gantt segments: pid, start, end
`p_result.csv`	CT, TAT, WT, RT (preemptive)
`p_gantt.csv`	Gantt segments (preemptive)
`comparison.csv`	Average WT, TAT, RT side by side

2.2 Computed Metrics

For each process i:

```
CT[i]  = Completion Time
TAT[i] = CT[i] - AT[i]           (Turnaround Time)
WT[i]  = TAT[i] - BT[i]          (Waiting Time)
RT[i]  = first_start[i] - AT[i]  (Response Time)
```

Average values are computed by `Metrics::fillAverages` after scheduling.

2.3 Tie-Breaking Rule

When multiple processes share the same priority: **lower arrival time wins**. When arrival times are equal: **earlier position in the input CSV wins**. This rule applies to both algorithms and is tested explicitly.

2.4 Key Challenges

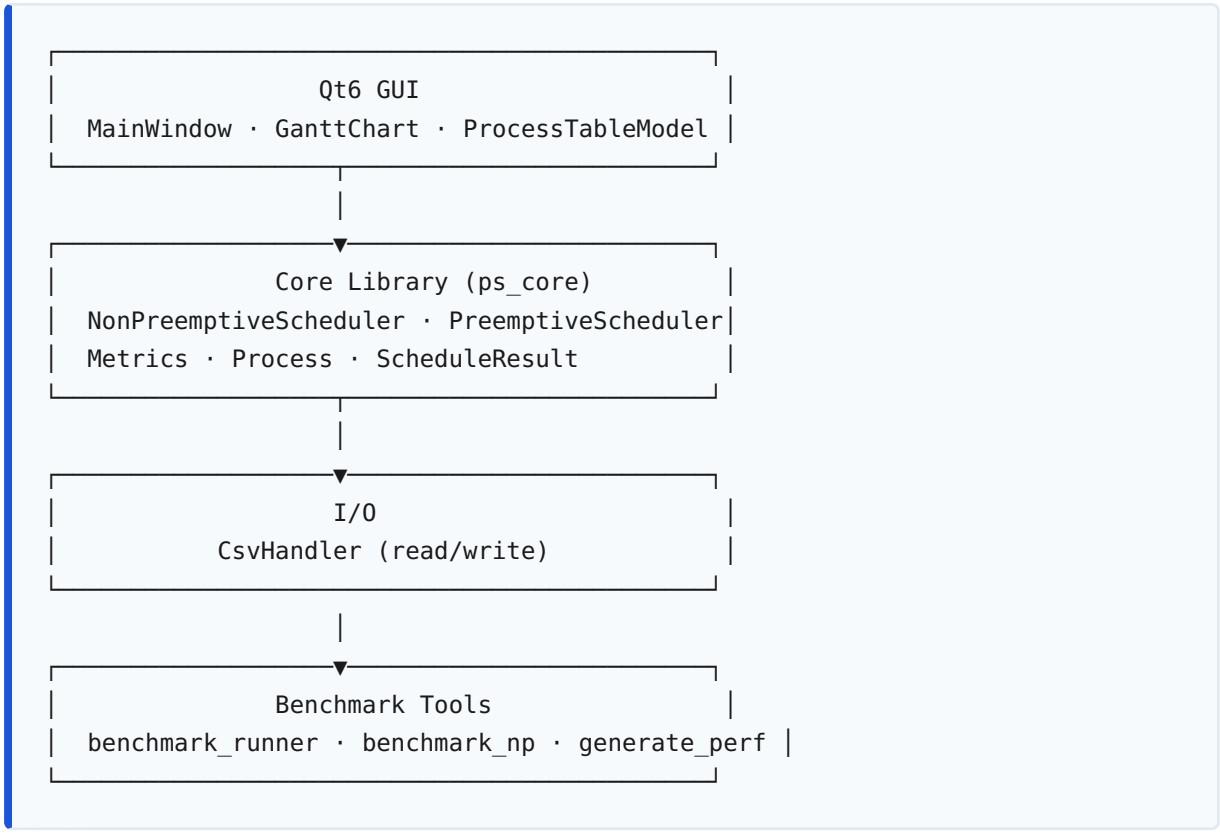
Challenge	Solution
Non-preemptive $O(n^2)$ with naive list scan	Min-heap + FIFO per priority level → O(n log n)
Preemptive tick-by-tick simulation is O(n × T)	Event-based simulation, jump to next event → O(n log n)
GUI table with 1,000,000 rows freezes	Virtual `QTableView` + `ProcessTableModel` (only render visible rows)
Gantt chart unreadable with many segments	`QScrollArea` + minimum 24 px per segment

---

3. System Design

3.1 Architecture

The system is divided into four independent layers:



Dependency rule: `core` does not include Qt or I/O. `io` does not include Qt. `ui` may include all. This keeps both schedulers testable without Qt.

3.2 Core Data Structures

```
struct Process {
    int       pid;           // > 0, unique
    long long arrivalTime;   // AT >= 0
    long long burstTime;     // BT > 0
    int       priority;      // lower = higher priority
};

struct GanttSegment {
    int       pid;           // -1 = IDLE
    long long startTime;
    long long endTime;
};

struct ScheduleResult {
    std::vector<ProcessResult> processes;  // same order as input
    std::vector<GanttSegment>  gantt;
    double averageWaitingTime    = 0.0;
    double averageTurnaroundTime = 0.0;
    double averageResponseTime   = 0.0;
};
```

All time fields use `long long` to support arrival times up to 10^{12}.

3.3 Module Responsibilities

Module	Class / Target	Responsibility
`src/core/`	`NonPreemptiveScheduler`	Schedule without preemption
`src/core/`	`PreemptiveScheduler`	Schedule with preemption
`src/core/`	`Metrics`	Compute and fill averages
`src/io/`	`CsvHandler`	Read/validate CSV, write 5 output files
`src/ui/`	`MainWindow`	UI orchestration
`src/ui/`	`GanttChart`	Paint Gantt segments inside `QScrollArea`
`src/ui/`	`ProcessTableModel`	Virtual model for `QTableView`
`src/benchmark/`	`Benchmark`	Shared benchmark utilities (timing, stats, RSS wrapper)
`src/benchmark_main.cpp`	`benchmark_runner`	Measure PreemptiveScheduler
`tools/benchmark_np.cpp`	`benchmark_np`	Measure NonPreemptiveScheduler (5 runs)
`tools/generate_perf.cpp`	`generate_perf`	Generate benchmark CSV

---

4. Implementation

4.1 Non-preemptive Priority Scheduler

Algorithm (pseudocode):

```
function schedule(processes):
    sort indices by (AT, inputIndex)
    ready = min-heap ordered by (priority, AT, inputIndex)
    time = 0

    while uncompleted processes remain:
        if ready is empty:
            append IDLE segment [time, next_arrival]
            time = next_arrival

        load all processes where AT <= time into ready
        i = ready.pop()                    // highest priority
        start = time
        time += BT[i]
        record CT, TAT, WT, RT for i
        append Gantt segment [start, time]
```

Key implementation decisions:

- Indices sorted by `(AT, inputIndex)` using `std::sort` with explicit tie-break key — no `stable_sort` needed.
- Ready queue uses `std::map<int, FIFO>` keyed by priority: `map` gives the minimum priority in O(log k), where k is the number of distinct ready priorities (typically small). Each priority level uses an intrusive linked array for O(1) FIFO access.
- This reduces selection cost compared to a global heap over all waiting processes.

Complexity: O(n log n + n log k), where k ≤ distinct priorities.
Memory: O(n).

4.2 Preemptive Priority Scheduler

Algorithm (pseudocode):

```
function schedule(processes):
    remaining[i] = BT[i] for all i
    start[i] = -1 for all i
    sort indices by AT (stable)
    ready = min-heap ordered by (priority, AT, inputIndex)
    time = 0

    while completed < n:
        load all arrivals with AT <= time into ready
        if ready is empty:
            jump to next arrival (IDLE segment)
            continue

        curr = ready.pop()
        eventTime = time + remaining[curr]

        for each upcoming arrival a with AT < eventTime:
            if priority[a] < priority[curr]:
                eventTime = AT[a]        // preempt at exact arrival
                break
            else:
                push a into ready        // higher number = lower priority, no
preempt

        remaining[curr] -= (eventTime - time)
        append/merge Gantt segment [time, eventTime]
        time = eventTime

        if remaining[curr] == 0:
            record CT, TAT, WT, RT
            completed++
        else:
            push curr back into ready   // preempted
```

Key implementation decisions:

- **Event-based simulation** (not tick-by-tick): the scheduler jumps directly to the next significant time point (next arrival or next burst completion). This eliminates the O(T) dependency on total burst time.
- **Arrival at exact eventTime boundary** uses strict `<` (not `≤`) so a process arriving exactly when the current one finishes is loaded in the next iteration — preventing zero-length Gantt segments.
- Adjacent same-PID Gantt segments are merged during emission.

Complexity: O(n log n + g), where g = number of Gantt segments.
Memory: O(n + g).

4.3 CSV Handler

`CsvHandler::readProcesses` validates:

- Header must be exactly `pid,arrival,burst,priority`
- Each field must parse as an integer
- `pid > 0` and unique; `burst > 0`; `arrival >= 0`
- `pid`, `priority` within `int` range; `arrival`, `burst` within `long long` range

Error collection: all invalid lines are collected (up to 100 errors) rather than stopping at the first. The error string is returned to the caller; `MainWindow` truncates to 10 lines before display.

4.4 GUI

MainWindow flow:

```
Open CSV → CsvHandler::readProcesses → display tableInput
Run → NonPreemptiveScheduler and/or PreemptiveScheduler
    → Metrics::fillAverages → update tableResult + averages + GanttChart
Export → CsvHandler::write* → output/
```

Virtual table: `tableInput` and `tableResult` use `QTableView` backed by `ProcessTableModel`, a `QAbstractTableModel` that generates cell strings on demand. It holds a pointer to the existing `std::vector<Process>` / `ScheduleResult` without copying — reset cost is O(1), display cost is proportional to visible rows only.

Gantt chart scrolling: `GanttChart` is wrapped in a `QScrollArea` (horizontal scroll only). The widget's minimum width is `segments × 24 px`, capped at 1,000,000 px (Qt's widget size limit is 16,777,215 px). When this exceeds the frame width, the scroll bar appears. The `paintEvent` only draws segments intersecting `event->rect()`, so scroll performance is independent of total Gantt width.

Run mode: the `comboMode` selector allows running Non-preemptive only, Preemptive only, or both with comparison.

---

5. Testing

5.1 Test Framework

All test suites use **doctest** (single-header, `tests/third_party/doctest.h`). Tests are compiled into three separate executables and registered with CTest.

```
cmake -S . -B build
cmake --build build
QT_QPA_PLATFORM=offscreen ctest --test-dir build --output-on-failure
```

5.2 Test Summary

Suite	Executable	Test Cases	Assertions	Scope
Unit	`unit_tests`	79	112,207	Schedulers, CSV, Metrics
Integration	`integration_tests`	10	318	Full pipeline: CSV → schedule → export
GUI	`gui_tests`	10	215	Headless Qt (offscreen)
Total	**ctest 3/3**	**99**	**112,740**	

5.3 Selected Test Cases

Non-preemptive (N01–N18):

ID	Description	Result
N01	Standard 10-process sample	Full Gantt + averages match expected CSV
N02	IDLE at start (first arrival > 0)	Correct IDLE segment
N04	Tie: same priority, same arrival	FIFO by input order
N09	Input with 2,000 processes	Correct output, < 1 s
N13	40 processes, all AT=0 — tie-break by index	Catches pid-based tie-break bug
N14	Process arrives exactly when CPU frees	No zero-length IDLE
N16	Priority INT_MIN and INT_MAX	No overflow in comparison
N17	IDLE segment lasting 10^{12} time units	No `long long` overflow

Preemptive (P01–P18):

ID	Description	Result
P01	Standard sample	Gantt includes preemption segments
P05	Arrival exactly at event boundary	No zero-length Gantt segment
P10	Same priority — no preemption	Running process completes
P15	30,000 processes	Correct output, < 2 s
P18	Stress: 500 processes random	Full Gantt coverage, correct averages

CSV (C01–C25):

ID	Description
C01	Valid 10-process file → 10 processes loaded
C03	Missing header → error message, 0 processes
C15	Multiple errors in one file → all errors reported
C22	500-process file → all loaded correctly

GUI (U01–U16):

ID	Description
U01	Load `gui/sample.csv` → table displays all rows in order
U03	Run NP → Gantt matches `ket_qua/gui/sample_np_gantt.csv`
U09	Export both → 5 files produced
U14	Load 10,005-row file → table shows all rows; Gantt scroll bar appears
U16	Load `sample.csv` (10 segments) → no scroll bar

Integration (I01–I10):

ID	Description
I01	Full pipeline: `sample.csv` → NP → result matches expected CSV
I02	Full pipeline: `sample.csv` → P → result matches expected CSV
I07	2,000-process stress: NP produces 2,001 Gantt segments, correct span

5.4 Correctness Strategy

- Preemptive results verified against an independent $O(n^2)$ oracle on 200 random inputs (fixed seed).
- Gantt continuity invariant (`[0, max(CT)]`, no gaps, no overlaps) checked in every scheduler test.
- Mutation testing on GUI result table: forcing the wrong row index turns 27 assertions red.

---

6. Performance Benchmarks

6.1 Environment

Item	Value
Machine	Intel Core i7-1185G7 @ 3.00 GHz (developer's Windows machine)
Verification machine	Intel Core i5-11400F, 15 GiB RAM, Ubuntu 24.04
Compiler	GNU C++ 16.2.0 / GCC 13.3.0
Build type	Release (`-O2 -DNDEBUG`)
Measurement	`std::chrono::steady_clock`, scheduler only (excludes CSV I/O)

6.2 Non-preemptive Scheduler

Measured with `benchmark_np <file> 5` (5 runs, mean and median reported).

Processes	Mean (ms)	Median (ms)	Peak RSS (MiB)
100,000	13.7	13.7	17
1,000,000	242	243	106
2,000,000	577	578	255

Optimization applied: replaced naive list scan ($O(n^2)$) with min-heap + FIFO-per-priority-level ($O(n \log n + n \log k)$). Speedup on 1M processes: ~3.4× (1,180 ms → 353 ms on developer machine; 242 ms on verification machine).

Targets set in `docs/NonPreemptive.md`: **NP 1M < 400 ms, 2M < 1,000 ms** — both met.

6.3 Preemptive Scheduler

Measured with `benchmark_runner <file>` (single run).

Processes	Time (ms)
100,000	27
200,000	64
500,000	242
1,000,000	572

Optimization: replaced tick-by-tick simulation (O(n × T) where T = total burst time, potentially billions) with event-based simulation (O(n log n)). The old implementation took 311 seconds for 100,000 processes; the new one takes 27 ms — a ~11,500× speedup.

6.4 GUI with 1,000,000 Rows

Measured on i5-11400F, offscreen 1180×900, Debug build.

Operation	Time
`loadCsv` (read + parse)	272 ms
`runSchedules` (NP only)	251 ms
`runSchedules` (both NP + P)	835 ms
Window grab (screenshot)	21 ms
Scroll to last row	1 ms
Peak RSS	247 MiB

The virtual table model and event-based Gantt painting make 1,000,000-row display feasible without a worker thread.

---

7. Results and Discussion

7.1 Sample Run — 10 Processes

Input (`input/sample.csv`):

PID	AT	BT	Priority
1	0	5	3
2	1	3	2
3	2	8	1
4	3	2	4
5	4	4	5
6	5	1	1
7	6	3	3
8	7	2	2
9	8	5	1
10	9	2	4

Non-preemptive Gantt:

`P1[0–5] → P3[5–13] → P6[13–14] → P2[14–17] → P4[17–19] → P8[19–21] → P10[21–23] → P7[23–26] → P5[26–30] → P9[30–35]`

Metric	NP	P
Avg Waiting Time	8.00	7.80
Avg Turnaround Time	11.50	11.30
Avg Response Time	8.00	4.70

Observation: Preemptive scheduling achieves lower average waiting and response time because high-priority processes that arrive late immediately preempt lower-priority ones. However, it generates more Gantt segments and is slightly slower to compute due to additional heap operations.

7.2 Algorithm Comparison

Property	Non-preemptive	Preemptive
Preemption	None	Immediate on higher-priority arrival
Response time	Higher (must wait for running process)	Lower (gets CPU immediately)
Starvation risk	Yes (low-priority processes may wait indefinitely)	Yes (more severe — can be interrupted repeatedly)
Complexity	O(n log n)	O(n log n)
Gantt segments	At most n + IDLE gaps	Up to $O(n^2)$ in worst case
Implementation	Simpler	Event-based, more complex

7.3 Limitations

- Both algorithms are susceptible to starvation of low-priority processes. No aging mechanism is implemented (out of scope for this project).
- Priority values are plain integers; no support for dynamic priority adjustment.
- Benchmark data is generated with a fixed random seed; real-world workloads may differ.

---

8. Conclusion

This project successfully implements and compares Non-preemptive and Preemptive Priority CPU scheduling:

1. **Correctness** is validated by 99 test cases and 112,740 assertions across unit, integration, and GUI test suites. All 3 ctest suites pass on both Debug and Release builds.
2. **Performance** meets the stated targets: NP handles 1,000,000 processes in 242 ms and 2,000,000 in 577 ms; P handles 1,000,000 in 572 ms. Both achieve O(n log n) behavior through min-heap + event-based design.
3. **Usability** is addressed by the Qt6 GUI: virtual table display for unlimited row counts, horizontal Gantt scroll, one-click export of 5 CSV files, and clear error messages for invalid input.

4. **Maintainability** is supported by a frozen API contract (`docs/api.md`), a strict dependency rule (core ← io ← ui), and comprehensive documentation in `docs/`.

---

9. Member Contributions

9.1 Checkpoint 02 — Team Progress (Milestones)

Week	Workstream	Deliverables	Status
1	Project foundation	CMake, conventions, frozen API contract	Done
2	CSV module	`CsvHandler`, CSV validation + unit tests (C01–C25)	Done
3	Non-preemptive Priority	`NonPreemptiveScheduler` + tests (N01–N18)	Done
4	Preemptive Priority	`PreemptiveScheduler` + tests (P01–P18)	Done
5	GUI skeleton + integration	`MainWindow`, `GanttChart`, GUI wiring + tests (U01–U13)	Done
6	GUI scalability	Virtual `QTableView` model for very large datasets + horizontal-scroll Gantt	Done (merge PR #40)
7	Benchmark + report data	`benchmark_runner`, `benchmark_np`, `generate_perf`, performance logs	Done
8	Verification + final docs	`ctest` suites, final report + demo reference	Code ready; report finalized

Verification snapshot (at the time of this report): - `ctest`: 3/3 suites passed - Unit tests: 79 test cases / 112,207 assertions - Integration tests: 10 test cases / 318 assertions - GUI tests: 10 test cases / 215 assertions

9.2 Member Responsibility

Member	Module	Deliverables
Hưng (Leader)	Architecture, build, integration	`CMakeLists.txt`, `docs/api.md`, `docs/conventions.md`, integration tests (I01–I10), benchmark tools (`benchmark_np`, `generate_perf`), `report/non-preemptive-optimization.md`
Kiệt NP	Non-preemptive scheduler	`src/core/NonPreemptiveScheduler.{h,cpp}`, unit tests N01–N18 (`np_regression_test.cpp`, `np_edge_test.cpp`)
Vĩ P	Preemptive scheduler	`src/core/PreemptiveScheduler.{h,cpp}`, unit tests P01–P18 (`preemptive_test.cpp`)
Minh Hưng GUI	Qt GUI	`src/ui/MainWindow.{h,cpp}`, `src/ui/GanttChart.{h,cpp}`, `src/ui/ProcessTableModel.{h,cpp}`, GUI tests U01–U16 (`gui_test.cpp`)
Lan CSV	CSV I/O	`src/io/CsvHandler.{h,cpp}`, unit tests C01–C25 (`csv_handler_test.cpp`)

Appendix A — Build Instructions

```
# Linux
sudo apt install qt6-base-dev cmake g++ git
git clone https://github.com/Hhanguu/PriorityScheduling.git
cd PriorityScheduling
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j4

# Run application
./build/PriorityScheduling

# Run all tests
QT_QPA_PLATFORM=offscreen ctest --test-dir build --output-on-failure

# Generate benchmark data and measure
./build/generate_perf --out input --sizes 100000 1000000
./build/benchmark_runner input/perf_100000.csv
./build/benchmark_np input/perf_1000000.csv 5
```

Appendix B — Repository Structure

```
src/
  core/      Process.h, GanttSegment.h, ScheduleResult.h, Metrics.h
             NonPreemptiveScheduler.{h,cpp}, PreemptiveScheduler.{h,cpp}
  io/        CsvHandler.{h,cpp}
  ui/        MainWindow.{h,cpp}, GanttChart.{h,cpp}, ProcessTableModel.{h,cpp}
  benchmark/ Benchmark.{h,cpp}
  main.cpp
  benchmark_main.cpp
tools/
  generate_perf.cpp, benchmark_np.cpp
tests/
  unit/      csv/, non-preemptive/, preemptive/, leader/
  integration/
  gui/
  data/      input CSVs + ket_qua/ (expected outputs)
  third_party/doctest.h
docs/
  api.md, algorithms.md, conventions.md, plan.md,
  NonPreemptive.md, Preemptive.md, GUI.md, csv-handler.md,
  test-cases/
input/       sample.csv, sample_20.csv (perf_*.csv gitignored)
output/      5 sample result CSVs
report/      non-preemptive-optimization.md, report.md
```

Appendix C — Invariants

The following invariants are enforced by both schedulers and verified by the test suite:

#	Invariant
I1	`ScheduleResult.processes` preserves input order
I2	Gantt covers `[0, max(CT)]` with no gaps and no overlaps
I3	No two adjacent Gantt segments have the same PID
I4	IDLE is represented by `pid = -1`
I5	Schedulers do not fill `average*` fields; `Metrics` does
I6	Empty input → empty `ScheduleResult`, no crash
I7	`WT = TAT − BT`, `TAT = CT − AT`, `RT = first_start − AT`
I8	No context switch overhead, no I/O modeled